



TP 1 — Java RMI

Année 2026–2027 Enseignant Bachar Rima

OBJECTIF

Construire, de bout en bout, une application distribuée en **Java RMI natif** : un cabinet vétérinaire dont les patients vivent dans la JVM du serveur et sont manipulés à distance par des clients. Le CM 02 a posé le modèle, le TD 1 l'a fait lire ; il s'agit maintenant de l'écrire.

Chaque étape pose une **décision de conception** — cet objet doit-il rester chez lui et être appelé à distance, ou bien traverser le réseau sous forme de copie ? — et se termine par un **résultat observable** qui prouve qu'elle fonctionne.

PRÉREQUIS

- **CM 01** — les cinq patrons des intergiciels : Proxy, Adapter, Factory, Interceptor, Observer.
- **CM 02** — interface distante, servant, export, registre, stub, et surtout les *trois régimes de passage* de la section 5.
- **TD 1** — lecture de l'exemple `helloWorld` : ordre de démarrage, rôle du registre, ce que reçoit réellement le client.
- Bases de Java et de la programmation orientée objet (voir la ressource de rappel ci-dessous si nécessaire).

1. Ressources fournies

CODE DE RÉFÉRENCE

- **hello-rmi-single-project** — l'exemple `helloWorld` du TD 1, exécutable tel quel. À lancer en premier pour vérifier votre environnement.
- **hello-rmi-multi-project** — la même application découpée en trois projets `common` / `server` / `client`. **C'est la structure attendue pour ce TP.**

Chaque dossier contient un `README.md` avec les commandes de compilation et d'exécution, et ce que vous devez observer.

ANNEXES

- **Annexe « RMI : workflow et aide graduée » (2026–2027)** — rappels d'architecture, ordre de lancement, erreurs fréquentes et comment les lire.
- **Annexe « RMI avec Spring Boot » (2026–2027)** — nécessaire uniquement pour la partie B.
- **Annexe « Spring Boot Crash Course » (2026–2027)** — support général de l'UE : framework, inversion de contrôle, injection de dépendances, beans, `ApplicationContext`, cycle de vie d'une application non web. Utile pour la partie B et pour les TP suivants.
- **Annexe « Patrons utiles aux architectures distribuées » (2026–2027)** et [resources/design-patterns](#) — les cinq patrons du CM 01, chacun avec un exemple exécutable. Le patron *Observer* éclaire l'étape A6.

BESOIN D'UN RAPPEL JAVA/POO ?

Si les bases du langage ou de la conception objet ne sont pas encore consolidées, deux listes de lecture sont à votre disposition : [Java/POO — partie 1](#) et [partie 2](#).

2. Environnement technique

ÉLÉMENT	EXIGENCE POUR CE TP
JDK	Java 17 au minimum. Le sujet est validé sur 17.0.10 et 21.0.5 (les deux versions LTS visées par le CM 02).
Projets	non modulaires , sur le <i>classpath</i> . Ne créez pas de <code>module-info.java</code> : le système de modules (JPMS) est hors sujet ici, et sa configuration vous ferait perdre du temps sans rien vous apprendre sur RMI.
Outils RMI	<code>rmiregistry</code> , livré avec le JDK.
Interface	une CLI, avec la seule bibliothèque standard de Java (<code>Scanner</code> , <code>BufferedReader</code>) : aucune dépendance.
Partie B	Maven et une connexion réseau pour la première résolution des dépendances (voir la partie B, page 7).
Partie C	Swing (inclus dans le JDK) ou JavaFX, à ajouter au projet (voir la partie C, page 9).

Environnement de développement. Il est **fortement recommandé** d'utiliser un environnement de développement intégré (IDE), notamment **Eclipse** ou **IntelliJ IDEA**. La maîtrise d'un IDE — navigation dans le code, exécution de plusieurs processus, débogueur — constitue une compétence utile pour ce TP, pour la suite du Master et pour le développement logiciel professionnel.

Il reste tout à fait possible de réaliser le TP avec un **éditeur de texte/code et les outils en ligne de commande** : Visual Studio Code, Sublime Text, Emacs, Vim, Xcode ou tout autre environnement équivalent, combinés à `javac`, `java` et `rmiregistry`. Aucun élément du sujet ne dépend d'un IDE particulier, et les deux approches sont évaluées de la même façon.

NOTE

Quel que soit votre choix, vous devez être capable d'expliquer *ce que votre outil fait à votre place* : où sont rangées les classes compilées, ce que contient le *classpath* de chaque projet, et quel processus est lancé quand vous cliquez sur « Run ». C'est exactement ce que demande le README à produire.

3. Modalités de travail

Travail individuel. Le TP 1 se réalise et se rend **seul**. (Le travail en binôme sera autorisé à partir du TP 2 ; les modalités seront précisées à ce moment-là.) Discuter d'un problème avec un camarade est normal et encouragé ; rendre un code que vous ne sauriez pas expliquer ne l'est pas.

3.1. Usage des outils d'intelligence artificielle générative

L'usage d'outils d'IA générative est **autorisé**, à condition qu'il reste un *appui* et qu'il soit *déclaré*.

USAGES LÉGITIMES	CE QUI N'EST PAS ACCEPTABLE
Faire expliciter une notion du cours.	Faire produire tout ou partie de la solution sans la comprendre.
Demander des exemples supplémentaires.	Rendre du code que vous ne sauriez pas justifier.
Comprendre un message d'erreur, chercher les causes possibles d'un bogue.	Présenter comme vôtre une conception que vous n'avez pas examinée.
Discuter des alternatives de conception, organiser des idées, améliorer la rédaction d'une explication, localiser une documentation.	Produire un compte-rendu dont le contenu ne correspond pas à votre code ou à votre vidéo.

Vous restez **entièrement responsables** de l'architecture, de l'implémentation, de la correction, de la vérification et des explications que vous rendez. Vous devez pouvoir **expliquer et défendre** chaque élément significatif de votre travail.

CE SUR QUOI PORTE RÉELLEMENT L'ÉVALUATION

Il n'existe aucun moyen fiable de détecter automatiquement du texte ou du code engendré par une IA, et ce TP ne cherche pas à le faire. L'évaluation porte sur la **transparence**, la **compréhension démontrée**, la **cohérence entre le compte-rendu, le code et la vidéo**, votre capacité à **justifier vos choix** et la **reproductibilité** de votre travail. Un dossier cohérent et assumé, avec une déclaration honnête, n'est jamais pénalisé par le fait d'avoir utilisé un assistant.

Le compte-rendu doit comporter une section dédiée : voir § 4, point **A.9**.

PARTIE A Java RMI « vanilla »

OBLIGATOIRE · COMPTE POUR LA NOTE SUR 20

Cette partie est la **totalité du travail requis**. Elle se construit étape par étape : chaque étape ajoute une pièce à l'application précédente et se clôt par un point de contrôle observable. **N'attaquez pas une étape tant que le point de contrôle de la précédente n'est pas franchi** — c'est la façon la plus économique de déboguer une application distribuée.

SCÉNARIOS DE TEST ET APPLICATION FINALE

Pendant les étapes A1 à A7, il est normal — et recommandé — de valider chaque fonctionnalité par quelques appels écrits directement dans un `main`. Mais **les appels codés en dur dans `main()` constituent des scénarios de test, pas une interface utilisateur**. L'application rendue doit être *utilisable* par quelqu'un qui n'a ni à lire ni à recompiler votre code : c'est l'objet de l'étape A8, qui dote le client d'une interface en ligne de commande. Une interface graphique est possible en bonus (partie C).

A0 Mise en route et vérification de l'environnement

TRAVAIL DEMANDÉ

1. Vérifiez votre version de Java : `java -version` doit annoncer **17 ou plus**.
2. Compilez et exécutez `hello-rmi-single-project` en suivant son `README.md`.
3. Recommencez avec `hello-rmi-multi-project`, dans les deux modes de registre (externe et interne).
4. Créez la structure de votre propre application, sur le même modèle : trois projets `common`, `server`, `client`, les deux derniers ayant `common` sur leur `classpath`.

POINT DE CONTRÔLE 0

Le client affiche `response: Hello, world!` tandis que `The server prints : Hello, world!` apparaît **dans la console du serveur**. Vos trois projets compilent, et le projet `client` ne référence **aucune** classe du projet `server`.

A1 Un premier objet distant : l'animal

Chaque patient du cabinet est un animal, caractérisé au minimum par un nom, le nom de son maître, sa race et son espèce. Dans cette première étape, l'espèce n'est encore qu'une simple chaîne de caractères.

TRAVAIL DEMANDÉ

1. Dans `common`, définissez le **contrat distant** de l'animal : une interface qui étend `java.rmi.Remote`, dont chaque méthode déclare `RemoteException`.
2. Dans `server`, écrivez le **servant** correspondant et exportez-le.
3. Écrivez le lanceur du serveur : il crée un animal, obtient un registre et publie l'objet sous un nom.
4. Écrivez un client qui retrouve l'animal et invoque quelques méthodes d'affichage.

INDICE

Deux façons d'obtenir un registre : le lancer à part avec `rmiregistry` (`LocateRegistry.getRegistry()`), ou le créer dans la JVM du serveur (`LocateRegistry.createRegistry(1099)`). Les deux sont acceptées. Préférez `rebind` à `bind` pendant le développement (TD 1, question 7).

POINT DE CONTRÔLE A

Depuis un **processus client séparé**, vous obtenez l'animal et affichez son nom. Affichez également `stub.getClass().getName()` : vous devez lire un nom de la forme `jdk.proxyN.$ProxyM`, et `Proxy.isProxyClass(...)` doit renvoyer `true`.

A2 L'espèce : une donnée qui traverse le réseau

L'espèce devient une **classe** à part entière (par exemple un nom et une espérance de vie moyenne). On veut pouvoir *consulter* l'espèce d'un animal, mais pas la modifier à distance.

TRAVAIL DEMANDÉ

1. Faites de l'espèce un objet **transmis par valeur**. Où doit vivre cette classe pour que le client *et* le serveur la connaissent ?
2. Modifiez l'animal pour qu'il expose son espèce.
3. Côté client, récupérez l'espèce d'un animal, **modifiez la copie locale**, puis redemandez l'espèce au serveur.

POINT DE CONTRÔLE B

Affichez `System.identityHashCode(...)` de l'espèce des deux côtés : les valeurs diffèrent. Après votre mutation locale, le serveur renvoie toujours la valeur d'origine. **Il y a bien deux objets Java distincts** — c'est la différence entre « envoyer une photo de l'objet » et « envoyer un moyen de le joindre » (CM 02, section 5).

A3 Le dossier de suivi : à vous de décider

Chaque animal possède un **dossier de suivi** : au minimum son état de santé, et de préférence un historique d'observations. Plusieurs vétérinaires — donc plusieurs clients — doivent pouvoir le **consulter et le modifier**.

TRAVAIL DEMANDÉ

C'est la décision de conception centrale du TP.

1. Tranchez : le dossier de suivi est-il un **objet distant** (il reste sur le serveur, on l'appelle à distance) ou un **objet sérialisable** (il est copié chez le client) ?
2. Implémentez votre choix, et permettez au client de lire *et* de modifier le dossier d'un animal.
3. **Justifiez votre décision dans le compte-rendu**, en vous appuyant sur le comportement observé.

INDICE

Posez-vous la question du partage : si deux clients modifient « le » dossier, que doit voir le troisième ? Puis demandez-vous laquelle des deux sémantiques produit ce comportement. Rien ne vous interdit d'implémenter les deux pour comparer.

POINT DE CONTRÔLE C

Un client modifie l'état du dossier d'un animal; une **seconde lecture** (par exemple en redemandant l'animal au serveur), montre la modification. Si ce n'est pas le cas, votre dossier est copié — ce qui peut être un choix, à condition de l'assumer et de l'expliquer.

A4 Le cabinet vétérinaire : un point d'entrée unique

Publier chaque animal individuellement dans le registre ne passe pas à l'échelle : il faudrait connaître à l'avance le nom de chaque patient. On introduit donc un **cabinet vétérinaire**, qui connaît sa collection de patients et devient le seul objet publié.

TRAVAIL DEMANDÉ

1. Définissez le contrat distant du cabinet. Il doit au minimum permettre d'obtenir la liste des patients et d'en **rechercher un par son nom**.
2. Le serveur ne publie plus que le cabinet.
3. Adaptez le client : il récupère le cabinet, puis navigue vers les animaux.

INDICE

Que retourne `getPatients()` du point de vue du client : des copies d'animaux, ou des moyens de les joindre ? La réponse découle de ce que vous avez décidé à l'étape A1, et c'est un bon test de cohérence de votre conception.

POINT DE CONTRÔLE D

Le registre ne contient plus qu'**une seule liaison**. Le client affiche la liste des patients et retrouve un animal par son nom; une recherche infructueuse se comporte de manière définie et documentée (valeur nulle, exception métier... à vous de choisir et de le dire).

A5 Créer un patient depuis le client

Jusqu'ici les patients étaient créés par le serveur à son démarrage. Un vétérinaire doit pouvoir **enregistrer un nouveau patient** depuis son poste.

TRAVAIL DEMANDÉ

1. Ajoutez au cabinet une opération d'ajout de patient, callable à distance.
2. Décidez de la signature : quels paramètres traversent le réseau, et sous quelle forme ? Notamment, comment l'espèce est-elle transmise ?
3. Vérifiez le résultat **par une interaction distante**, et non par une trace côté serveur.

INDICE

Où l'objet « animal » doit-il être instancié pour être un objet distant utilisable ? Un `new` exécuté dans la JVM du client ne crée pas un objet « chez le serveur ».

POINT DE CONTRÔLE E

Après l'appel, le nombre de patients renvoyé par le serveur a augmenté, et une **recherche par nom** depuis le client retrouve le nouveau patient.

A6 Alertes : le serveur rappelle ses clients

On souhaite que **tous les vétérinaires connectés** soient prévenus automatiquement lorsque le nombre de patients franchit les seuils **100, 500 et 1000**, à la hausse comme à la baisse. L'alerte se traduit par un affichage sur la sortie standard *du client*.

TRAVAIL DEMANDÉ

1. Définissez, dans `common`, le contrat de l'**observateur**.
2. Côté client, implémentez-le et **exportez** cet objet : le client devient à son tour hébergeur d'un objet distant.
3. Ajoutez au cabinet l'abonnement et le désabonnement, et déclenchez les notifications au franchissement des seuils.
4. Testez avec **au moins deux clients abonnés simultanément**.

INDICE

C'est le patron **Observer** réalisé à travers le réseau (CM01 patron 5; CM02, section 5). L'annexe « Patrons utiles » et [resources/design-patterns/observer](#) en donnent la version *locale*; à distance, l'observateur doit être *exporté*, c'est son *stub* que reçoit le serveur, et chaque notification devient un appel distant qui peut échouer. Question de robustesse : que se passe-t-il si un client abonné a été fermé entre temps ? Votre serveur doit y survivre.

POINT DE CONTRÔLE F

Deux clients sont abonnés ; le franchissement du centième patient provoque un affichage **dans les deux consoles clientes**. Fermer brutalement un client n'empêche pas l'autre de continuer à recevoir ses alertes.

A7 Ce qui traverse vraiment le réseau

Une courte expérience sur la sérialisation.

TRAVAIL DEMANDÉ

Expérience. Dans le projet **client** uniquement, définissez une sous-classe de votre classe « espèce » (par exemple `Chien`). Envoyez-en une instance au serveur via l'opération d'ajout de patient de l'étape A5.

1. Observez l'échec, et **lisez la cause profonde** de l'exception.
2. Expliquez-la : qu'est-ce qu'un flux sérialisé contient exactement, et qu'est-ce qu'il ne contient pas ?
3. Corrigez, de la manière la plus simple possible.
4. Consignez l'expérience, l'explication et la correction dans le compte-rendu.

POINT DE CONTRÔLE G

Avant correction, vous obtenez une `ClassNotFoundException` portant le nom complètement qualifié de votre sous-classe, encapsulée dans une `UnmarshalException` puis une `ServerException`. Après correction, le patient est ajouté normalement.

A8 Une application utilisable : l'interface en ligne de commande

Les fonctionnalités existent ; il faut maintenant qu'un vétérinaire puisse s'en servir. Le client final expose une **interface en ligne de commande** (CLI) qui donne accès aux fonctionnalités de l'application **sans modifier ni recompiler le code**.

TRAVAIL DEMANDÉ

1. Depuis la CLI, l'utilisateur doit pouvoir :
 - consulter la liste des patients et rechercher un patient par son nom ;
 - enregistrer un nouveau patient, espèce comprise ;

- consulter le dossier de suivi d'un patient et le mettre à jour;
- s'abonner aux alertes et s'en désabonner, les alertes s'affichant pendant l'utilisation;
- quitter proprement.

2. Organisez le client en **trois responsabilités distinctes** (les noms sont libres) :

point d'entree	lit les arguments (hote, port), obtient le stub, lance l'interface
logique client	les operations du client, sur les interfaces distantes
interface console	affiche le menu, lit et valide les saisies, affiche les resultats

L'interface console ne doit contenir aucun appel à `java.rmi` : elle s'appuie sur la logique client.

3. Traitez les saisies invalides et les échecs distants par un message compréhensible, sans arrêter le client.
4. Utilisez la bibliothèque standard (`Scanner` ou `BufferedReader`).

INDICE

Deux questions à vous poser en testant. Une alerte peut arriver pendant que l'utilisateur est en train de saisir : où et comment s'affiche-t-elle ? Et après avoir choisi « quitter » en étant abonné aux alertes, votre processus client se termine-t-il vraiment ? Si non, qu'est-ce qui maintient la JVM en vie ?

POINT DE CONTRÔLE H – APPLICATION UTILISABLE

Le client démarre et affiche son menu. Depuis **deux clients CLI** lancés dans des terminaux séparés, vous enregistrez un patient, le retrouvez, modifiez son dossier et voyez la modification depuis l'autre client ; les deux reçoivent une alerte. Une saisie invalide produit un message, pas une exception. Arrêter le serveur puis invoquer une opération produit un message intelligible. « Quitter » termine le processus client.

VALIDATION FINALE

- ☐ L'application démarre à partir d'un dépôt propre, en suivant votre seul README.
- ☐ Elle tourne en **processus séparés** : registre (le cas échéant), serveur, et au moins deux clients CLI.
- ☐ Les huit points de contrôle (0, A à H) sont franchis.
- ☐ Aucune fonctionnalité ne nécessite de modifier le code pour être utilisée.
- ☐ Les cas d'échec prévisibles sont gérés : registre absent, nom non lié, client injoignable, recherche infructueuse, saisie invalide.

PARTIE B Java RMI piloté par Spring Boot

BONUS OPTIONNEL · NON NÉCESSAIRE POUR TERMINER LE TP

LISEZ CECI AVANT DE COMMENCER

Cette partie est **entièrement facultative**. Elle n'est **pas nécessaire** pour terminer le TP, et son absence ne coûte **aucun point** : le barème sur 20 ne porte que sur la partie A. Ne l'abordez que si la partie A est terminée et validée.

Son intérêt : les TP suivants de l'UE utiliseront Spring Boot de façon plus systématique. C'est ici une première prise de contact, sans enjeu.

L'idée n'est **pas** de tout réécrire, mais de **migrer** votre application de la partie A :

PARTIE A

« Je construis moi-même toute l'infrastructure RMI. »

PARTIE B

« Je garde le mécanisme RMI, mais je laisse Spring gérer l'assemblage, le cycle de vie et la configuration. »

CE QUE LA PARTIE B N'EST PAS

Spring Boot n'implique **pas** une application web. Vous n'ajouterez ni Spring Web, ni contrôleur REST, ni Tomcat : la seule dépendance nécessaire est `spring-boot-starter`. REST sera étudié en CM 04 et pratiqué au TP 3. Vous n'utiliserez pas non plus `RmiServiceExporter` ni `RmiProxyFactoryBean` : tout le paquetage `org.springframework.remoting` a été **retiré de Spring Framework 6**, donc de toute version actuelle de Spring Boot. Le RMI reste **natif** — `LocateRegistry`, `UnicastRemoteObject`, `rebind`, `lookup`.

B1 Créer les projets Spring Boot

Deux applications Spring Boot (serveur et client) et un module partagé, avec Maven. Version épinglée : **Spring Boot 4.1.1, Java 17**. L'annexe donne les quatre `pom.xml` exacts : le projet parent et ses trois modules.

B2 Isoler le contrat dans un module commun

Les interfaces distantes et les types transmis par valeur vont dans un module Maven `vet-common`, dont dépendent le serveur et le client. C'est votre projet `common` de la partie A, devenu un artefact.

B3 Faire du serviant un bean

Le cabinet devient un composant géré par le conteneur (`@Service`). Il continue d'étendre `UnicastRemoteObject` et d'implémenter votre interface distante.

B4 Injecter les dépendances par constructeur

Extrayez la logique métier — la collection de patients, par exemple — dans un bean séparé, injecté **par constructeur** dans le serviant.

B5 Créer et publier le registre depuis la configuration

Dans une classe `@Configuration`, exposez le registre RMI comme un `@Bean` (`LocateRegistry.createRegistry`), puis publiez le serviant au démarrage avec un `ApplicationRunner`. Le port et le nom du service se lisent dans `application.properties`.

B6 Obtenir le stub côté client par injection

Côté client, le `lookup` n'a plus lieu au milieu du code métier : il est fait une fois dans une `@Configuration`, et le stub est **injecté** dans votre logique client. La CLI de l'étape A8 est lancée depuis un `ApplicationRunner`, **sans être réécrite** : Spring assemble, il ne remplace ni l'interface ni la logique.

B7 Exécuter, puis comparer

Lancez le serveur, puis le client. Vérifiez que le registre est créé, le service publié, le stub obtenu, l'invocation distante effectuée — et que l'arrêt du serveur est propre.

LA QUESTION QUI COMPTE

Terminez par une demi-page dans le compte-rendu : **qu'est-ce que Spring Boot a simplifié, et quelle part reste du Java RMI?** Distinguez ce qui relève de l'assemblage et du cycle de vie de ce qui relève de la communication distante.

PARTIE C Une interface graphique

BONUS OPTIONNEL · NON NÉCESSAIRE POUR TERMINER LE TP

LISEZ CECI AVANT DE COMMENCER

Cette partie est **entièrement facultative**, comme la partie B. La CLI de l'étape A8 est l'interface **requis** ; une interface graphique (GUI) ne la remplace pas dans le barème sur 20, et son absence ne coûte aucun point. Ne l'abordez qu'une fois la partie A terminée.

L'objectif n'est pas de refaire le client, mais de **brancher une seconde interface sur la même logique client**. C'est le test le plus simple de la séparation des responsabilités demandée en A8 : si elle est bonne, la GUI n'a presque rien à savoir de RMI.

```
CLI (etape A8) ---+
                  +----> logique client ---> stub ---> serveur
GUI (partie C) ---+
```

CONDITION POUR OBTENIR LA TOTALITÉ DU BONUS

La GUI **réutilise** la logique client de l'étape A8, sans la dupliquer. Une interface graphique qui recopie les appels RMI, la gestion des erreurs ou l'abonnement aux alertes n'obtient qu'une partie du bonus (voir le barème).

C1 Choisir une technologie

Toute technologie Java défendable est acceptée, par exemple :

- **Swing** — inclus dans le JDK : aucune dépendance, aucun changement de configuration. C'est le choix le plus direct pour ce TP.
- **JavaFX** — n'est plus fourni avec le JDK depuis Java 11 : il faut ajouter les bibliothèques OpenJFX au projet (par exemple avec Maven), en suivant la documentation officielle openjfx.io. Prévoyez le temps de cette configuration.
- **Une interface web locale**, si elle ne fait que présenter les opérations de votre logique client : entre le client et le serveur, la communication reste RMI.

C2 Construire l'interface

Donnez accès aux principales opérations de l'étape A8 et affichez les alertes reçues. Deux points techniques :

- une alerte arrive sur un fil d'exécution du runtime RMI : la mise à jour de l'interface doit être confiée au fil graphique (`SwingUtilities.invokeLater`, `Platform.runLater`);
- un appel distant peut être lent ou échouer : il ne doit pas figer l'interface sans retour pour l'utilisateur.

C3 Démontrer et justifier

Montrez la GUI dans la vidéo, et expliquez en quelques lignes du compte-rendu comment elle réutilise la logique client.

4. Livrables et modalités de remise

Vous rendez **une seule archive**, nommée `NOM_Prenom_HAI704I_TP1.zip`, contenant les quatre livrables ci-dessous. Un livrable manquant, illisible ou inaccessible affecte directement les points correspondants du barème.

4.1. A — Compte-rendu (PDF)

Un **rapport d'ingénierie**, pas une documentation de code : il explique et justifie ce que vous avez fait. Ce n'est ni un listing complet des sources, ni de la Javadoc engendrée, ni un commentaire ligne à ligne.

Page de titre : université et formation, code et intitulé de l'UE (HAI704I — Architectures logicielles distribuées), numéro et titre du TP, enseignant responsable, année universitaire, date, puis vos **nom**, **prénom**, **numéro étudiant** et **parcours** (GL / IASD / ICO).

Contenu attendu :

1. une table des matières ;
2. un rappel bref des objectifs du TP ;
3. un compte rendu **étape par étape**, structuré en chapitres/sections/sous-sections — pas un récit continu ;
4. pour chaque étape : ce qui a été implémenté, la démarche adoptée, les **décisions de conception** et leur **justification**, et la façon dont le résultat a été validé ; pour l'étape A8, quelques paragraphes suffisent sur l'organisation de l'interaction avec l'utilisateur, la séparation entre interface, logique client et RMI, et le traitement des saisies invalides et des pannes distantes ;
5. les **schémas** qui clarifient réellement votre solution — diagramme de classes, de séquence, de composants, de déploiement, cas d'utilisation, activité... *Choisissez ceux qui apportent quelque chose* ; un diagramme produit mécaniquement ne rapporte rien ;
6. des **extraits de code choisis**, uniquement lorsqu'ils appuient une explication ;
7. une **synthèse** : ce qui fonctionne, ce qui ne fonctionne pas et pourquoi, les difficultés rencontrées, comment vous les avez cherchées et résolues — ou pourquoi elles ne le sont pas ;
8. un **regard critique** : ce qui resterait à améliorer (code, architecture, robustesse, maintenabilité, ergonomie, tests...) et comment ;
9. une section « **Usage des outils d'intelligence artificielle générative** » (voir § 3.1) indiquant : si vous en avez utilisé ; lesquels, aussi précisément que raisonnable ; pour quelles tâches ; le type de sollicitations ; ce qui a réellement influencé le rendu ; comment vous l'avez vérifié, corrigé ou adapté ; et ce qui est entièrement de vous. *Si vous n'en avez pas utilisé, dites-le explicitement*. Il n'est pas demandé de joindre un historique de conversation ;
10. les **références** : notes de bas de page pour un renvoi isolé, bibliographie/webographie pour les sources substantielles, dans un style de citation constant.

4.2. B — Code source

Les projets complets, permettant de compiler et d'exécuter l'application. **Sans** artefacts engendrés : pas de `target/` ni de `bin/` reproductibles, pas de caches d'IDE, pas de dépendances embarquées, pas de fichiers temporaires.

Si l'archive dépasse environ **50 Mo** — ou la limite affichée sur Moodle — vous pouvez fournir à la place un lien vers un **dépôt Git** (GitHub, GitLab ou équivalent stable). Dans tous les cas, **si vous utilisez un dépôt, son URL doit figurer dans le compte-rendu**, même lorsque les sources sont aussi dans l'archive. Évitez les services de partage de fichiers génériques ou temporaires ; si un hébergement externe est inévitable, assurez-vous que l'équipe pédagogique y a accès sans démarche particulière et que le lien reste valide pendant toute la période de correction.

4.3. C — README (PDF)

Un ou plusieurs README selon l'architecture de vos projets, rendus **en PDF** pour faire partie du dossier évalué. Vous pouvez évidemment conserver en plus un `README.md` dans votre dépôt.

Un lecteur qui ne connaît pas votre travail doit pouvoir **reconstituer et exécuter** l'application à partir de ce seul document :

LE PROJET	L'EXÉCUTION
rôle de chaque projet/module et relations entre eux	version exacte du JDK
structure des sources	commandes de compilation et de lancement
dépendances éventuelles	ordre de démarrage des processus
hypothèses de configuration	ports utilisés, mode de registre retenu
organisation du client (interface, logique, RMI)	lancement de la CLI , menu, conventions de saisie, arrêt propre
environnement de développement utilisé : IDE et version, ou éditeur + outils en ligne de commande	

4.4. D — Vidéo de démonstration

Une démonstration de l'application **finalisée, en cours d'exécution** — pas de diaporama. Elle doit montrer :

- le démarrage du registre, du serveur et des clients, dans cet ordre ;
- l'**utilisation interactive de la CLI** : plusieurs opérations principales déclenchées par l'utilisateur, et leur correspondance avec les étapes du TP ;
- les effets répartis : une modification faite depuis un client, observée depuis l'autre ; les alertes reçues ;
- l'arrêt propre des clients.

Un `main` qui déroule tout seul un scénario prédéfini montre un test, pas une application utilisable : il ne suffit pas. Si vous avez réalisé la partie B ou C, la vidéo peut aussi la montrer. Commentaire oral, sous-titres ou annotations à l'écran : au choix.

La vidéo peut être incluse dans l'archive si sa taille le permet, placée dans le dépôt, ou déposée sur une plateforme stable (YouTube ou équivalent). Si elle est hébergée à l'extérieur, **son URL doit apparaître dans le compte-rendu et dans le README** : le correcteur ne doit pas avoir à la chercher. Vous êtes responsable des droits d'accès.

DÉPÔT ET DATE LIMITE

Structure attendue de l'archive (adaptez-la si votre architecture le justifie) :

```
NOM_Prenom_HAI704I_TP1.zip
|-- Compte-Rendu.pdf
|-- README.pdf
|-- source/
|   |-- common/
|   |-- server/
|   |-- client/
|-- video/
|   |-- demonstration.mp4      (ou liens.txt si video externe)
|-- liens.txt                  (depot Git, video : facultatif si deja dans le CR)
```

Date limite de remise : consulter le dépôt Moodle correspondant à votre parcours.

Il existe **deux espaces de dépôt distincts**, l'un pour **GL/IASD**, l'autre pour **ICO**. Il vous appartient de déposer dans l'espace correspondant à votre parcours et d'y vérifier l'échéance ; tout ajustement ultérieur du calendrier y sera répercuté.

5. Barème

Le barème porte **uniquement sur la partie A** et totalise **20 points**. Chaque critère renvoie à une étape explicite du sujet.

CRITÈRE	ÉTAPES	CE QUI EST REGARDÉ	PTS
MISE EN ŒUVRE RMI – 8 points			
Objet distant publié et invoqué à distance	A1	contrat, export, publication, <i>lookup</i>	2
Cabinet : point d'entrée unique et recherche	A4	une seule liaison, navigation vers les patients	2
Création d'un patient depuis le client	A5	l'état est créé et conservé côté serveur	2
Alertes par rappel des clients	A6	observateur exporté par le client, seuils, plusieurs abonnés	2
CONCEPTION ET MAÎTRISE CONCEPTUELLE – 3 points			
Sémantique de passage justifiée	A2 A3 A7	valeur vs référence, décision assumée et expliquée	1,5
Architecture en trois projets	A0 – A8	le client ne dépend jamais de l'implémentation	1,5
APPLICATION UTILISABLE ET ROBUSTE – 3 points			
Interface en ligne de commande	A8	fonctionnalités accessibles sans modifier le code ; interface séparée de la logique client et de RMI	1,5
Comportement face aux défaillances	A1 – A8	exécution multi-processus réelle, saisies invalides, pannes distantes, arrêt propre	1,5
COMMUNICATION D'INGÉNIERIE ET REPRODUCTIBILITÉ – 6 points			
Compte-rendu	—	structure, justification, schémas utiles, synthèse et regard critique	2
Vidéo de démonstration	—	utilisation interactive de l'application finalisée	2
README	—	un tiers peut reconstituer, lancer et utiliser l'application	1
Déclaration d'usage de l'IA	—	présente, honnête, cohérente avec le rendu	1
Total			20

PARTIES B ET C – TRAITEMENT DES BONUS

Les parties B et C rapportent des points qui viennent **compenser** des points perdus sur la partie A. **Le total des bonus est plafonné à +2 points, et la note finale à 20/20.** La note de 20/20 reste accessible sans aucun bonus.

BONUS	PTS
Partie B – les projets Spring Boot compilent, le serveur publie le service, le client obtient le stub par injection et lance la CLI	+0,5
Partie B – la comparaison demandée en B7 est traitée sérieusement	+0,5
Partie C – une GUI utilisable donne accès aux principales opérations et aux alertes	+0,5
Partie C – la GUI réutilise la logique client de l'étape A8, sans la dupliquer	+0,5
Total pris en compte	+2 max.

Les bonus ne sont attribués que si l'application de la partie A fonctionne en processus séparés (points de contrôle A à E franchis).

CE QUI N'EST PAS ÉVALUÉ

La mise en forme décorative du compte-rendu, le nombre de schémas, la longueur du rendu, et le fait d'avoir ou non utilisé un assistant d'IA. À l'inverse, une incohérence entre le compte-rendu, le code et la vidéo affecte les critères correspondants.

6. Récapitulatif

À VÉRIFIER AVANT DE DÉPOSER

- ☐ Les huit points de contrôle (0, A à H) sont franchis.
- ☐ L'application tourne en processus séparés, avec au moins deux clients CLI.
- ☐ Toutes les fonctionnalités sont accessibles depuis la CLI, sans modifier le code.
- ☐ Aucun `module-info.java`.
- ☐ Le projet `client` ne référence aucune classe du projet `server`.
- ☐ `Compte-Rendu.pdf` – page de titre complète, table des matières, justification des choix, synthèse, regard critique, section IA, références.
- ☐ `README.pdf` – JDK, structure, build, ordre de lancement, ports, lancement et utilisation de la CLI, environnement de développement utilisé.
- ☐ Vidéo de démonstration montrant l'utilisation interactive de la CLI, accessible, URL dans le compte-rendu si externe.
- ☐ URL du dépôt Git dans le compte-rendu, le cas échéant.
- ☐ Archive nommée `NOM_Prenom_HAI704I_TP1.zip`, sans artefacts engendrés.
- ☐ Dépôt effectué dans **l'espace Moodle de votre parcours**, avant l'échéance qui y est affichée.